Name:- Suraj Singh Adhikari
Section:- NB-48
University Rollno:- 22515000139

Theory Assignment 3

1. What is the difference between frontend and backend JavaScript?

Frontend JavaScript runs inside the user's web browser and controls what the user sees and interacts with. It manipulates the DOM, handles events, validates forms and updates the page dynamically. It uses browser APIs such as `window`, `document` and `localStorage`. Frameworks: React, Angular, Vue.

Backend JavaScript runs on a server using a runtime such as **Node.js**. It handles business logic, databases, authentication, file systems and APIs. It has no DOM or `window`, but can use modules like `fs` and `http`. Frameworks: Express.js, NestJS.

Point	Frontend JS	Backend JS
Runs on	Browser (client side)	Server (Node.js)
Purpose	UI and user interaction	Logic, database, APIs
DOM access	Yes	No
File system access	No (restricted)	Yes
Examples	React, Vue, Angular	Express, NestJS

2. What is DOM?

DOM (**Document Object Model**) is a programming interface created by the browser for an HTML document. It represents the page as a **tree of objects (nodes)** where every element, attribute and piece of text is a node. JavaScript uses the DOM to read, change, add or delete elements, styles and content, which makes web pages dynamic.

```
document.getElementById("title").innerText = "Hello DOM";
document.querySelector("p").style.color = "blue";
```

3. What is event in JavaScript?

An **event** is an action or occurrence in the browser that JavaScript can detect and respond to, such as a click, key press, mouse move, form submit or page load. A function that runs in response is called an **event handler** (or listener), usually attached with `addEventListener()`.

```
const btn = document.getElementById("btn");
btn.addEventListener("click", function () {
  alert("Button clicked!");
});
```

4. What is event loop in JavaScript?

JavaScript is **single-threaded**, so it executes one task at a time. The **event loop** is the mechanism that lets it perform asynchronous work (timers, network requests, events) without blocking. It continuously checks whether the **call stack** is empty and, if so, moves the next pending callback into it.

- **Call Stack:** where functions are executed.
- **Web APIs:** browser features (setTimeout, fetch, DOM events) that handle async tasks.
- **Callback (Macrotask) Queue:** holds callbacks from timers and events.
- **Microtask Queue:** holds Promise callbacks; it has higher priority than the callback queue.
- **Event Loop:** moves tasks from the queues to the call stack when the stack is empty.

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("End");
// Output: Start, End, Promise, Timeout
```

5. What is hoisting in JavaScript?

Hoisting is JavaScript's behaviour of moving declarations of variables and functions to the top of their scope during the compile phase, before the code executes.

- `var` is hoisted and initialised with `undefined`.
- `let` and `const` are hoisted but not initialised (Temporal Dead Zone), so using them early gives a ReferenceError.
- Function declarations are fully hoisted and can be called before they are defined.

```
console.log(x);   // undefined
var x = 5;

greet();          // works
function greet() { console.log("Hi"); }

console.log(y);   // ReferenceError
let y = 10;
```

6. What is event bubbling?

Event bubbling is the process where an event triggered on an inner (child) element propagates upward through its ancestors: first the target element's handler runs, then its parent's, and so on up to the document. It is the default behaviour of most events. It can be stopped with `event.stopPropagation()`. The opposite is **event capturing**, where the event travels from the top down to the target. Bubbling also enables **event delegation**.

```
<div id="parent"><button id="child">Click</button></div>

parent.addEventListener("click", () => console.log("Parent"));
child.addEventListener("click",  () => console.log("Child"));
// Clicking the button prints: Child, then Parent
```

7. What is the difference between named function, anonymous function and arrow function?

- **Named function:** has a name, is hoisted (if a declaration), easy to debug and reusable.
- **Anonymous function:** has no name; usually assigned to a variable or passed as an argument (callback). Function expressions are not hoisted.
- **Arrow function:** shorter ES6 syntax using =>. It has no own `this`, `arguments` or `prototype`, takes `this` from the surrounding scope, and cannot be used as a constructor.

```
// Named function
function add(a, b) { return a + b; }

// Anonymous function
const sub = function (a, b) { return a - b; };
setTimeout(function () { console.log("done"); }, 1000);

// Arrow function
const mul = (a, b) => a * b;
```

8. Explain for-of and for-in loop?

for...in loops over the **enumerable property names (keys)** of an object. For arrays it gives the indexes (as strings). It is best suited to objects.

for...of loops over the **values** of iterable objects such as arrays, strings, Maps and Sets. It does not work on plain objects.

```
const arr = ["a", "b", "c"];
for (let i in arr) console.log(i);   // 0 1 2   (indexes)
for (let v of arr) console.log(v);   // a b c   (values)

const user = { name: "Suraj", age: 20 };
for (let key in user) console.log(key, user[key]);
```

9. Explain any five array methods?

- **push():** adds one or more elements at the end and returns the new length.
- **pop():** removes and returns the last element.
- **map():** creates a new array by applying a function to every element.
- **filter():** creates a new array with only the elements that pass a condition.
- **reduce():** reduces the array to a single value by applying a function with an accumulator.

```
let nums = [1, 2, 3, 4];
nums.push(5);                        // [1,2,3,4,5]
nums.pop();                          // removes 5 -> [1,2,3,4]
nums.map(n => n * 2);                // [2,4,6,8]
nums.filter(n => n % 2 === 0);       // [2,4]
nums.reduce((sum, n) => sum + n, 0); // 10
```

10. What is callback method?

A **callback** is a function that is passed as an argument to another function and is executed later, after some task completes or an event occurs. Callbacks are the basis of asynchronous programming in JavaScript (timers, events, API calls, array methods like `map` and `forEach`). Too many nested callbacks lead to 'callback hell', which is solved using Promises and async/await.

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}
greet("Suraj", function () { console.log("Callback executed"); });
```